

Distributed Cloud Architecture for Predictive Detection and Mitigation of Computer Vision Syndrome through Behavioral Analytics: General Architecture and Prototype Design

Andersson David Sánchez Méndez, Cristian Santiago Pedraza Rodríguez, Jeisson David Sánchez Gómez
Systems Engineering Program
Escuela Colombiana de Ingeniería Julio Garavito, Bogotá D.C., Colombia
{andersson.sanchez-m, cristian.pedraza-r, jeisson.sanchez-g}@mail.escuelaing.edu.co

Abstract

Computer Vision Syndrome (CVS), also known as Digital Eye Strain (DES), is a critical occupational health challenge affecting approximately 69% of the global population [15] and generating an estimated \$151 billion in combined productivity losses, direct healthcare costs, and well-being impacts in the United States alone during 2023 [16]. Despite growing awareness in both clinical and enterprise environments, existing mitigation tools remain fundamentally reactive, isolated at the device level, and architecturally incapable of supporting population-level analysis or proactive risk management at organizational scale. This article presents the complete general software architecture and the prototype-level design of a distributed, cloud-native platform for the continuous ingestion, processing, and machine-learning-based analysis of biometric and behavioral telemetry collected from edge devices. Building on the state-of-the-art analysis published in the first phase of this project, we formalize a four-layer architecture comprising an Edge Ingestion Layer, a Cloud Messaging Layer, an Asynchronous Processing and Persistence Layer, and a Machine Learning Inference Layer. Each layer is described through complementary architectural views aligned with the IEEE/ISO 42010 standard: context, component, data-flow, deployment, and security. The prototype is realized as a set of containerized microservices orchestrated with Kubernetes, a Kafka-based streaming pipeline, an InfluxDB time-series data store, and an XGBoost gradient-boosted CVS Risk Score inference module. We additionally present the alignment of the platform with the TOGAF Enterprise Architecture framework, formalize five Architecture Decision Records (ADRs), document the applied microservices design patterns (Circuit Breaker, CQRS, Database-per-Service, and Event Sourcing), and detail the CI/CD, observability, and data governance sub-architectures. Target Service Level Objectives for availability, scalability, security, and ML accuracy are defined and the evaluation methodology for the third project phase is specified.

Index Terms

Computer Vision Syndrome, Distributed Architecture, Cloud Native, Apache Kafka, Kubernetes, InfluxDB, Machine Learning, Microservices, IoT Telemetry, Federated Privacy, Enterprise Architecture, TOGAF, CQRS, Circuit Breaker, DevSecOps, Behavioral Analytics.

I. INTRODUCTION

The post-pandemic reconfiguration of knowledge work has produced an unprecedented intensification of digital screen exposure across the global workforce. Recent longitudinal surveys indicate that knowledge workers now accumulate, on average, 99.2 hours of screen time per week, with weekday exposure alone representing approximately 93% of waking hours [17]. These figures are not confined to a single geography or labor sector: a 2024 meta-analysis synthesizing data from 103 cross-sectional studies encompassing 66,577 participants determined that the pooled global prevalence of Computer Vision Syndrome (CVS) stands at approximately 69% of the working population [15]. The direct occupational consequence is substantial: 71% of affected desk workers report measurable productivity impacts attributable to CVS, with a mean productivity reduction of 18.6%—equivalent to roughly 7.4 lost productive hours per working week [17]. In aggregate, unmanaged CVS-related screen exposure incurred \$50.6 billion in productivity losses and over \$1.2 billion in direct healthcare system costs across the United States in 2023 [16].

From a software-engineering perspective, the root of the problem is not the absence of detection hardware. Modern smartphones are already equipped with proximity sensors, ambient light sensors, accelerometers, and high-resolution

front-facing cameras. The fundamental barrier is the **architectural inability** of current solutions to move beyond device-level information silos. Tools such as *Apple Screen Time*, *Google Digital Wellbeing*, or *f.lux* operate as closed-loop, reactive systems: they collect data locally, apply static usage thresholds, and generate local notifications, but they are structurally incapable of aggregating behavioral signals across users to build predictive population-level models. This architectural fragmentation prevents organizations from quantifying the “digital wear” of their human capital or from implementing evidence-based proactive interventions at the departmental or enterprise level.

The first phase of this research (State of the Art, First Tercio) established three foundational scientific challenges that any viable architecture must address: (i) energy- efficient edge sensing, (ii) behavioral data privacy and regulatory compliance, and (iii) elastic cloud scalability. It also introduced a four-layer architectural sketch. The present article advances those foundational contributions substantially. Its specific objectives are:

- 1) To formally specify the **general architecture** of the CVS mitigation platform through five complementary architectural views: context, component, data-flow, deployment, and security.
- 2) To present the **TOGAF-aligned enterprise architecture** view, mapping the four TOGAF domains (Business, Data, Application, Technology) to the platform’s structural components.
- 3) To detail the **prototype architecture** through concrete technology stack selections, Avro data schemas, Kafka topic designs, InfluxDB data models, REST API contracts, and ML pipeline specifications, formally documented as Architecture Decision Records (ADRs).
- 4) To document the **applied microservices design patterns** that increase the platform’s resilience, consistency, and observability.
- 5) To specify the **CI/CD pipeline, observability stack, and data governance framework** that together constitute the operational architecture of the prototype.
- 6) To define **target quality attributes** and Service Level Objectives (SLOs) that will be evaluated empirically in the third and final project phase.

The remainder of the article is organized as follows. Section II provides the problem characterization and architectural scope. Section III presents an extended related work analysis. Section IV introduces the TOGAF Enterprise Architecture alignment. Section V presents the General Architecture through five views. Section VI describes the Prototype Architecture in full technical detail. Section VII documents the applied microservices design patterns. Section VIII details the CI/CD pipeline and DevOps architecture. Section IX presents the observability and monitoring architecture. Section X covers the data governance and compliance framework. Section XI formalizes the Architecture Decision Records. Section XII discusses quality attributes and SLOs. Section XIII presents an economic impact analysis. Section XIV covers the risk analysis and mitigation strategies. Section XV outlines the evaluation methodology for the third phase. Section XVI concludes the article.

II. PROBLEM CHARACTERIZATION AND ARCHITECTURAL SCOPE

A. Clinical and Occupational Context

CVS encompasses a range of ocular and musculoskeletal symptoms that manifest from sustained digital screen interaction. Physiologically, the primary trigger is a reduction in the spontaneous blink rate from a neurological baseline of 15–20 blinks per minute to as few as 4–6 blinks per minute during active cognitive screen use [2]. Reduced blinking disrupts the stability of the precorneal tear film, leading to dry eye, asthenopia (eye fatigue), blurred vision, and accommodative dysfunction. Compounding risk factors include screen-to-eye distances below 30 cm, mismatches between ambient illuminance and screen brightness (measured in lux), and high-cognitive-load tasks that further suppress blink frequency [3], [18].

A 2024 controlled study demonstrated that CVS symptom severity scores begin to worsen measurably from baseline as early as 10 minutes into continuous screen use, and that the effect magnitude is significantly amplified under higher cognitive load conditions [18]. This temporal granularity has a direct architectural implication: any effective monitoring system must collect data *continuously* at sub-minute intervals rather than at the session level.

B. Architectural Gap Analysis

Table I summarizes the principal architectural gaps identified across the three solution categories reviewed in the first phase of this project. Each row also identifies the specific response this architecture provides to the identified gap.

Table I
ARCHITECTURAL GAP ANALYSIS OF EXISTING CVS MITIGATION APPROACHES

Category	Representative Tools	Core Limitation	Gap Addressed Here
Edge-Only Well-being	Apple Screen Time, Google Digital Wellbeing, f.lux	No cross-device learning; static thresholds; no population baselines [1]	Federated ML; population risk profiling; organizational dashboards
CV-Based Detection	CNN blink-rate monitors via webcam	High compute cost; privacy rejection due to continuous video recording; not viable as background service [4]	Non-visual proxy telemetry: ambient light, proximity, interaction time
Healthcare IoT	Fog-computing cardiac monitors [5]	Not generalized to visual ergonomics; no enterprise EA alignment	SOA on cloud; TOGAF-aligned four-domain architecture; compliance layer

C. System Boundary and Stakeholders

The system boundary encompasses five principal actor categories:

- **End User (Knowledge Worker):** installs the mobile agent; provides explicit informed consent; receives break-reminder push notifications.
- **Organizational Health Officer (OHO):** accesses aggregated, anonymized risk dashboards for workforce-level intervention planning and occupational health reporting.
- **IT / DevOps Administrator:** manages Kubernetes cluster health, CI/CD pipeline configuration, and cloud infrastructure provisioning.
- **ML Engineer:** curates anonymized training datasets, monitors model performance drift, and triggers retraining pipelines.
- **Regulatory / Compliance Officer:** audits GDPR compliance, data retention policies, and differential privacy configurations throughout the data lifecycle.

III. EXTENDED RELATED WORK

A. Clinical Epidemiology of Digital Eye Strain

The clinical literature on CVS has grown substantially since the early 2000s. Blehm et al. [3] provided a seminal review covering the biomechanical mechanisms of asthenopia, establishing the 30 cm minimum screen-to-eye distance and the role of ambient lux mismatch as ergonomic thresholds. Sheppard and Wolffsohn [2] updated these findings with a 2018 systematic review reporting CVS prevalence between 50% and 90% among computer users, and quantifying the blink-rate reduction from 15–20 to 4–6 blinks per minute as the primary pathophysiological driver. More recently, Talens-Estarellles et al. [18] provided controlled experimental evidence that CVS symptoms become clinically significant within 10 minutes of continuous screen use and are strongly modulated by cognitive load. This convergent body of evidence justifies the continuous, 30-second sampling interval adopted in our edge agent design, as opposed to session-level aggregation strategies used by commercial tools.

B. Non-Visual Proxy Sensing for Fatigue Detection

The use of non-visual behavioral signals as proxies for fatigue states has been explored extensively in the mobile sensing literature. Lane et al. [4] provided a foundational survey of mobile phone sensing capabilities, demonstrating that accelerometers, proximity sensors, and ambient light sensors collectively encode rich behavioral context even without camera access. This work is directly relevant to our architecture’s choice to rely exclusively on non-visual telemetry, avoiding the documented privacy rejection rates associated with continuous camera operation.

More recently, Alshurafa et al. [19] demonstrated the viability of wearable sensor networks for multi-level fatigue prediction in manufacturing workers, achieving competitive F1-scores using gradient-boosted tree classifiers on tabular biometric features rather than deep neural networks. Their finding that lightweight tree ensembles outperform neural networks on small-feature tabular datasets directly motivates our choice of XGBoost for the CVS Risk Score model. Chae et al. [22] extended this paradigm to visual fatigue specifically, demonstrating that classification models based on physiological responses to office lighting conditions can accurately predict visual fatigue onset, supporting the inclusion of ambient lux as a primary telemetry signal.

C. Cloud and Fog Computing for Healthcare IoT

Mutlag et al. [5] demonstrated how Fog Computing combined with microservices can reduce end-to-end latency for critical cardiac telemetry to clinically acceptable levels, and identified the architectural tradeoffs between edge-local and cloud-central processing. Their work introduced the concept of heterogeneous computational tiers—edge, fog, and cloud—that directly influenced our Layer 1 (edge buffering) and Layer 2 (cloud gateway) design.

Mahmoud et al. [6] provided a 2024 survey of IoT application architectures, classifying existing platforms along the dimensions of data flow, computational tier, and communication protocol. Their analysis confirms that the *publish-subscribe* message-broker pattern (exemplified by Kafka) outperforms both polling and synchronous REST for high-throughput IoT ingestion workloads, while providing the decoupling necessary for independent scalability of producer and consumer components.

D. Privacy-Preserving Machine Learning in Healthcare

Berneker et al. [13] conducted a scoping review of federated machine learning as a privacy-enhancing technology in medical research, mapping its compatibility with European data protection law. Their analysis determined that federated learning—where model training occurs locally on each institution’s data without raw data centralization—satisfies GDPR requirements under the data minimization and purpose limitation principles. Ali et al. [14] extended this analysis to the IoT healthcare context, presenting architectural patterns for federated learning integration in smart healthcare platforms. Both works substantiate our ADR-004 designation of federated learning as the long-term evolution path for the model training pipeline.

Alotaibi et al. [11] formalized the use of differential privacy with privacy budget $\epsilon \approx 1.0$ in real-time healthcare AI systems, demonstrating that this configuration satisfies GDPR requirements while maintaining statistical utility losses below 5 percentage points relative to unperturbed data. Mutlag et al. [23] confirmed these findings in the context of federated learning for healthcare, providing the mathematical framework for the Laplace noise mechanism we implement in the Anonymizer Microservice.

E. Time-Series Databases for IoT Telemetry

The choice between specialized time-series databases for high-throughput IoT telemetry has been a subject of active benchmarking. Comparative analyses [7] consistently demonstrate that InfluxDB outperforms TimescaleDB on pure write-heavy IoT ingestion workloads where queries are predominantly time-range aggregations, due to its purpose-built line protocol and native compaction strategies optimized for monotonically increasing timestamps. TimescaleDB retains advantages when workloads require complex relational JOIN operations with non-time-series data. Our platform’s write-dominant, aggregation-focused access pattern—collecting three sensor readings every 30 seconds per device across thousands of endpoints—aligns squarely with InfluxDB’s performance advantage profile.

F. Kubernetes for Healthcare Platform Orchestration

Boda and Allam [20] provided empirical evidence from production healthcare deployments of Kubernetes-orchestrated microservices, documenting improvements in resource utilization efficiency and system availability relative to VM-based deployments. Their findings are particularly relevant to the Horizontal Pod Autoscaler (HPA) configuration in our platform: they observed that setting CPU utilization targets between 60% and 70% provides the best tradeoff between cost efficiency and scale-out responsiveness, consistent with the 60% and 70% thresholds specified in our prototype’s Helm values.

IV. ENTERPRISE ARCHITECTURE FRAMEWORK ALIGNMENT

A. TOGAF ADM Mapping

The Open Group Architecture Framework (TOGAF) provides a globally recognized methodology for designing, implementing, and governing enterprise IT architecture through its Architecture Development Method (ADM) [24]. The ADM consists of a preliminary phase and eight iterative development phases (A through H), covering architecture vision, business architecture, information systems architecture, technology architecture, opportunities and solutions, migration planning, implementation governance, and change management. TOGAF has been adopted by 80% of Global 50 companies and 60% of Fortune 500 companies, and its application to complex healthcare IoT systems has been validated in multiple academic case studies [25].

Table II maps the CVS platform’s development activities to the corresponding TOGAF ADM phases, demonstrating formal alignment with enterprise architecture best practice.

Table II
TOGAF ADM PHASE MAPPING FOR THE CVS MITIGATION PLATFORM

TOGAF ADM Phase	Primary Output	CVS Platform Artifact
Preliminary	Architecture principles; governance model	Stakeholder map; system boundary definition
A: Architecture Vision	Stakeholder requirements; high-level solution concept	Problem characterization; four-layer sketch (First Tercio article)
B: Business Architecture	Business capability model; organizational context	OHO intervention workflow; consent management process
C: Information Systems	Data and application architecture	InfluxDB schema; Kafka topic design; REST API specification
D: Technology Architecture	Technology stack; deployment topology	Kubernetes deployment view; Helm chart; Istio service mesh config
E: Opportunities & Solutions	Implementation roadmap; prototype identification	Prototype technology stack selection; ADR formalization
F: Migration Planning	Prioritized implementation plan	Phase 2 (current) → Phase 3 evaluation roadmap
G: Implementation Governance	Architecture conformance reviews	SLO definitions; evaluation methodology (Section XV)
H: Change Management	Architecture change triggers	Model drift detection; Federated Learning migration path

B. TOGAF Four Architecture Domains

TOGAF defines four primary architecture domains, each representing a critical dimension of enterprise architecture [26]. Table III maps each domain to the corresponding CVS platform components.

Table III
TOGAF FOUR-DOMAIN MAPPING FOR THE CVS PLATFORM

TOGAF Domain	Domain Focus	CVS Platform Realization
Business Architecture	Strategy, governance, organizational processes	OHO intervention workflows; consent management process; GDPR compliance governance; organizational risk reporting cadence
Data Architecture	Data assets, governance, integration, and analytics	InfluxDB time-series schema; Avro message schema; differential privacy pipeline; 90-day data retention policy; GDPR Article-by-Article compliance map
Application Architecture	Application blueprint; interactions; business process alignment	Four-layer microservices architecture; REST API contract; Kafka topic topology; Kong API Gateway; Grafana analytics dashboard
Technology Architecture	Cloud infrastructure; platforms; standards	Kubernetes EKS cluster; Istio service mesh; Prometheus/Grafana observability; GitHub Actions CI/CD; Helm chart packaging

C. Architecture Repository and Governance

Following TOGAF’s Architecture Repository concept, all architectural artifacts produced by this project are organized into four categories: (i) Architecture Metamodel (component diagram, data schema, API specification); (ii) Architecture Capability (SLO definitions, quality attribute scenarios); (iii) Architecture Landscape (context view, deployment view, security view); and (iv) Standards Library (GDPR compliance mapping, IEEE/ISO 42010 viewpoints). These artifacts are maintained in the project’s GitHub repository under a version-controlled documentation structure that supports TOGAF’s change management phase [27].

V. GENERAL ARCHITECTURE

This section presents the platform’s general architecture through five complementary views: the Context View, the Component View, the Data-Flow View, the Deployment View, and the Security Architecture View.

A. Context View (Level 0)

Figure 1 illustrates the system context diagram at Level 0. The CVS Cloud Platform interacts with five external entities: knowledge workers transmitting telemetry via the mobile agent, an organizational health dashboard consuming risk data, an external identity provider for authentication, a notification gateway for push alerts, and DevOps administrators managing the infrastructure.

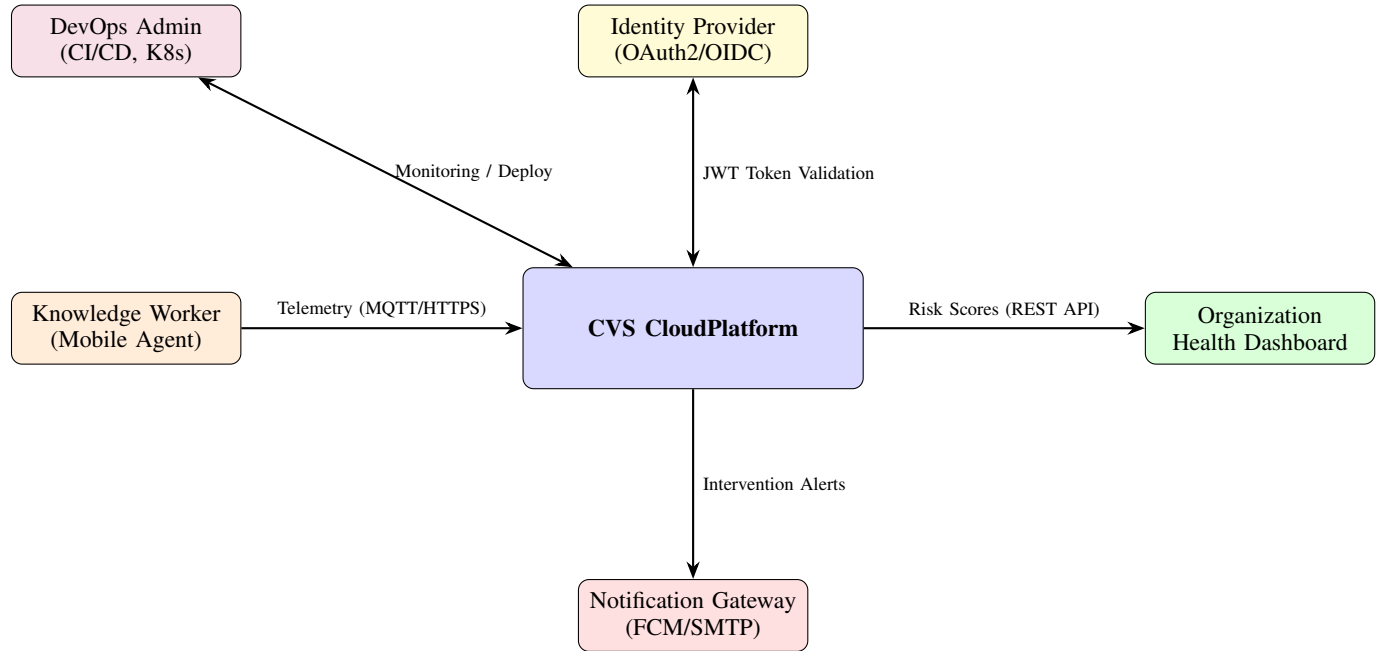


Figure 1. Level-0 Context Diagram: system boundary and external actor interactions.

B. Four-Layer Component Architecture (Level 1)

The platform's internal structure is organized into four horizontal processing layers, each implementing a well-defined separation of concerns. This design follows the layered architectural pattern, which reduces coupling between producers and consumers of telemetry data and provides independent scalability for each processing stage [6]. Figure 2 presents the Level-1 component diagram.

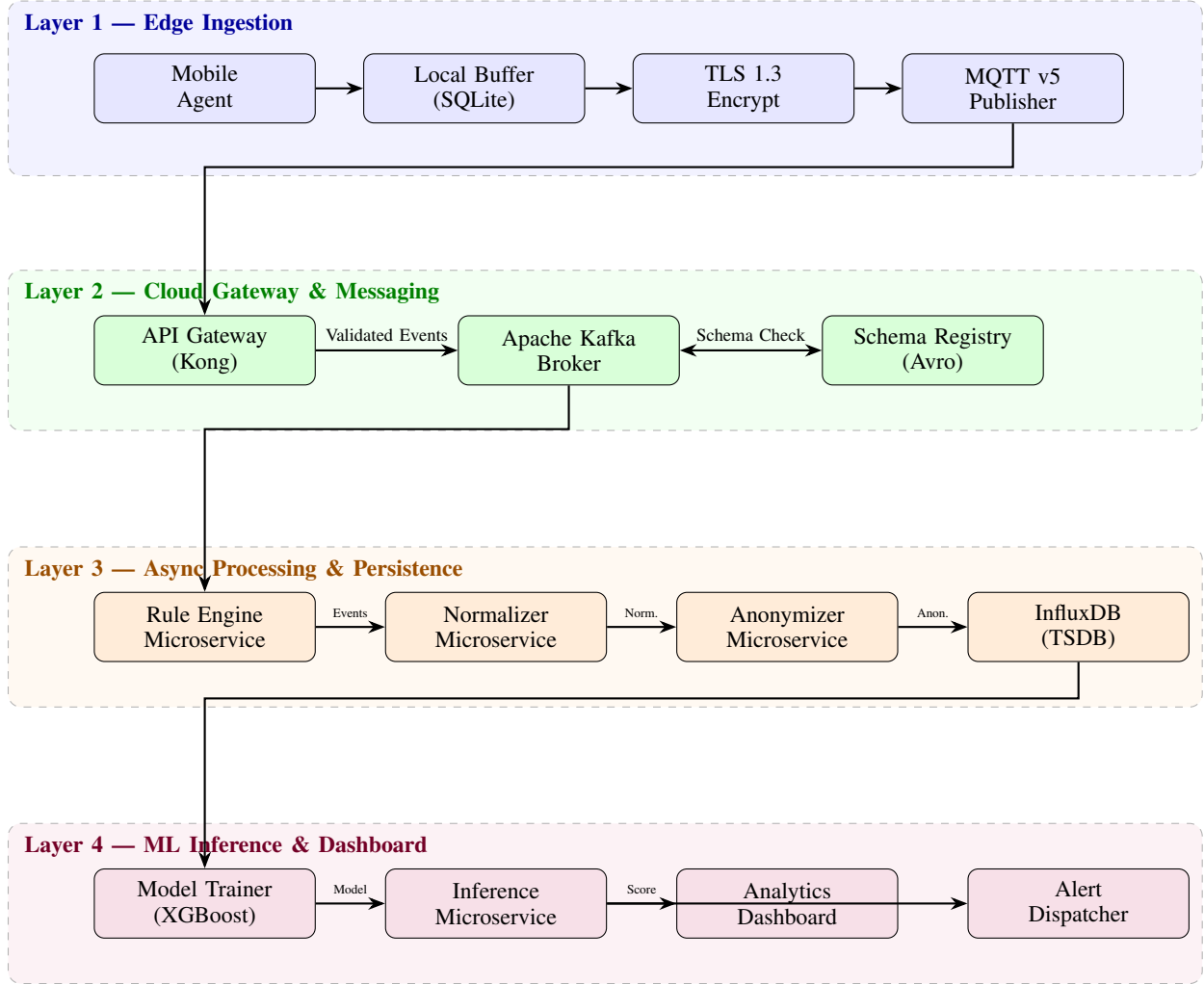


Figure 2. Level-1 Four-Layer Component Diagram of the CVS Cloud Platform.

C. Layer Responsibilities

1) *Layer 1 — Edge Ingestion*: The mobile agent is implemented as a battery-efficient background service. It polls native device APIs at a configurable sampling frequency (default: one reading every 30 seconds) for three primary signals: (i) ambient light level in lux from the device’s Ambient Light Sensor (ALS); (ii) screen-to-device proximity distance in centimeters from the proximity/ToF sensor; and (iii) active screen interaction time derived from the OS’s UsageStatsManager (Android) or CMMotionActivity (iOS) APIs. Readings are buffered locally in a SQLite micro-database and transmitted in micro-batches every 5 minutes over TLS 1.3-encrypted MQTT v5 channels. This micro-batch aggregation strategy preserves battery life while maintaining the temporal granularity required for the ML feature engineering pipeline [8].

2) *Layer 2 — Cloud Gateway and Messaging*: Inbound telemetry micro-batches arrive at a Kong API Gateway instance responsible for JWT authentication validation, rate limiting (maximum 100 requests/min per device), and request routing [9]. Authenticated payloads are forwarded to a 3-replica Apache Kafka broker cluster organized into partitioned topics. Kafka’s distributed commit log provides sub-millisecond latency at millions-of-events-per-second throughput, while fully decoupling producer and consumer lifecycles [10]. A Confluent Schema Registry enforces Avro message schemas, preventing schema drift from breaking downstream consumers.

3) *Layer 3 — Asynchronous Processing and Persistence*: Three stateless microservices consume Kafka topics in parallel consumer groups: (i) the Rule Engine Microservice applies immediate 20-20-20 clinical threshold evaluations; (ii) the Normalizer Microservice applies z-score standardization per sensor modality with linear forward-fill for missing readings; and (iii) the Anonymizer Microservice replaces device identifiers with pseudonymous

UUIDs and injects Laplace-distributed differential privacy noise ($\epsilon = 1.0$) to all continuous sensor fields [11]. Processed records are persisted to InfluxDB organized into buckets, measurements, tags, and fields.

4) *Layer 4 — Machine Learning Inference*: A scheduled model trainer executes nightly as a Kubernetes CronJob, retrieving the previous 30 days of anonymized records from InfluxDB, engineering a 9-feature matrix, and training a Gradient Boosted Trees classifier (XGBoost 2.0) to produce a CVS Risk Score $\in [0, 1]$ for each user demographic profile. The serialized model artifact is versioned in an S3-compatible object store. The stateless Inference Microservice computes real-time scores as new telemetry arrives. The Alert Dispatcher triggers Firebase Cloud Messaging (FCM) push notifications when a user’s rolling 60-minute risk score exceeds 0.70.

D. Data-Flow Diagram

Figure 3 presents the end-to-end data flow annotated with the transformation applied at each stage.

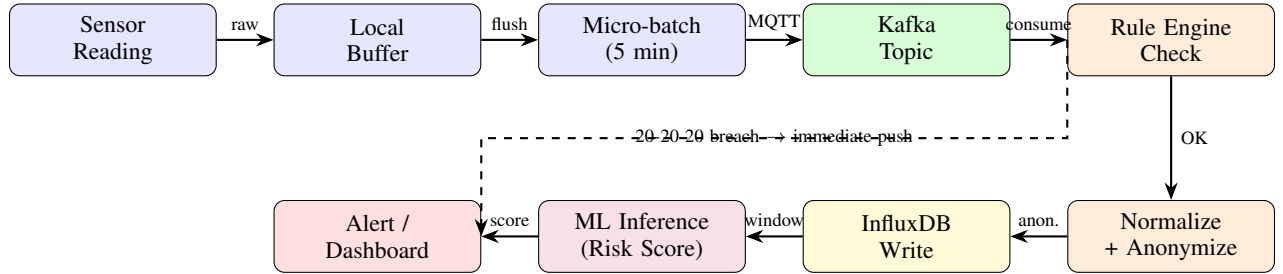


Figure 3. End-to-End Data-Flow: from raw sensor reading to organizational intervention.

E. Deployment View — Cloud Infrastructure

The platform is deployed on a managed Kubernetes cluster (AWS EKS or GCP GKE) organized into three node pools: an *ingestion pool* for API Gateway and Kafka Connect workers, a *processing pool* for stateless microservices, and a *storage pool* of SSD-backed persistent volumes for InfluxDB and the model artifact store.

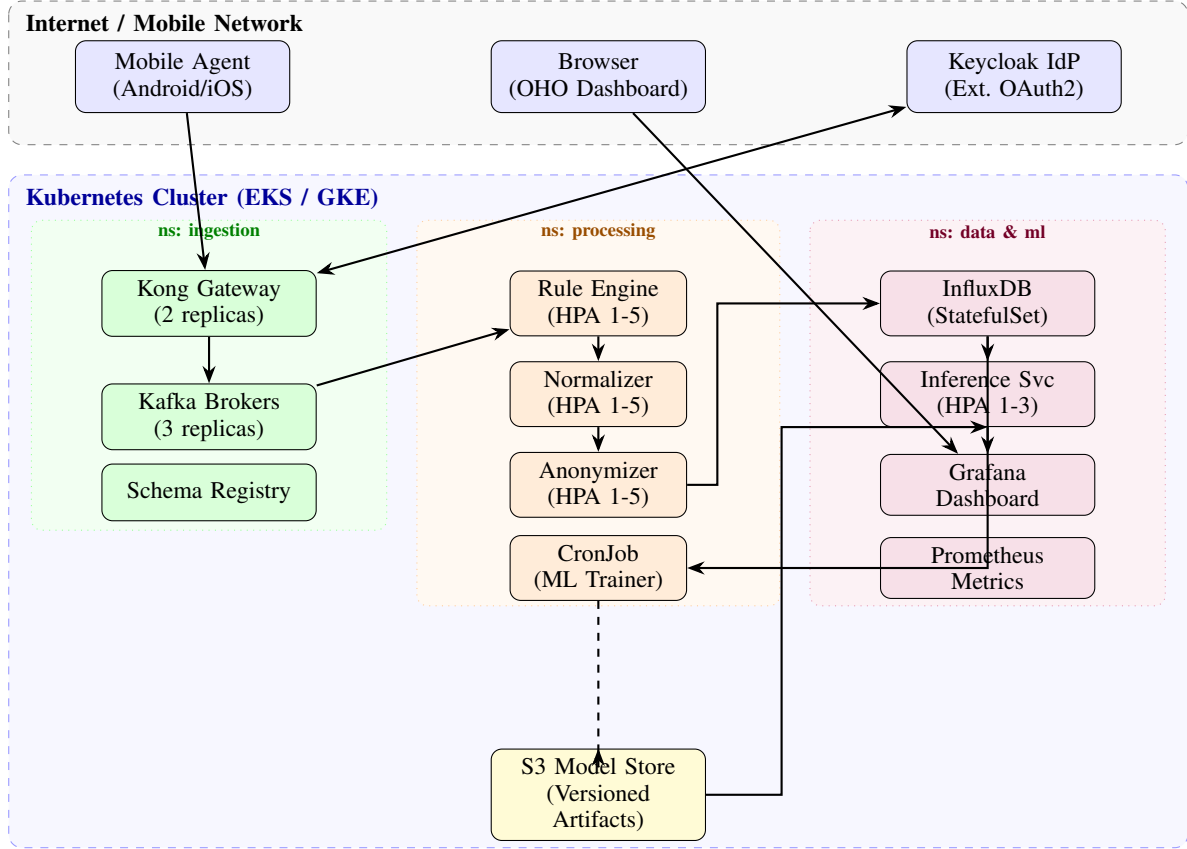


Figure 4. Deployment View: Kubernetes namespaces, node pools, and external dependencies.

F. Security Architecture View

Security is treated as a cross-cutting concern enforced at four independent layers. Figure 5 illustrates the layered security enforcement model.

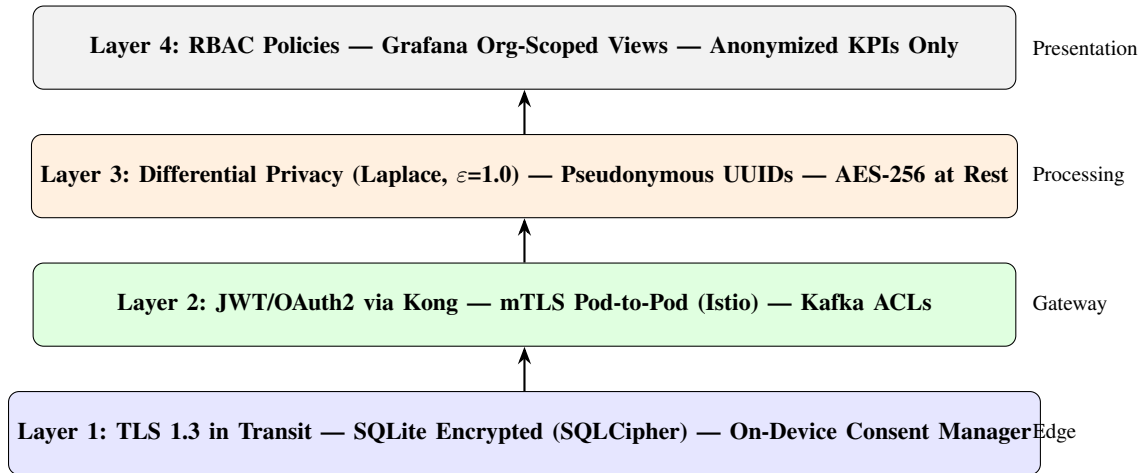


Figure 5. Layered Security Architecture: enforcement points across all four platform layers.

The security design is grounded in three regulatory principles. First, GDPR Article 5 mandates data minimization and purpose limitation [12]: the platform collects only the three non-invasive sensor signals strictly necessary for CVS risk estimation. Second, privacy-by-design is enforced at the Anonymizer Microservice through Laplace- distributed

noise with $\varepsilon = 1.0$, consistent with the GDPR-compliance framework established for healthcare AI by Alotaibi et al. [11]. Third, Federated Learning is designated as the long-term evolution of the model pipeline, keeping raw data entirely within institutional boundaries [13], [14].

VI. PROTOTYPE ARCHITECTURE

A. Technology Stack Summary

Table IV consolidates the technology selections for each component with the primary technical justification for each choice.

Table IV
PROTOTYPE TECHNOLOGY STACK

Component	Technology	Primary Justification
Mobile Agent	Kotlin + WorkManager	Battery-efficient background scheduling; guaranteed micro-batch execution without foreground service
Transport Protocol	MQTT v5 (Mosquitto)	Lightweight IoT publish-subscribe; QoS-1 guaranteed delivery; minimal overhead per message [8]
API Gateway	Kong Gateway 3.x	Plugin-based JWT validation, rate limiting, request transformation; native Kubernetes Ingress support [9]
Message Broker	Apache Kafka 3.6	Sub-ms latency; billions events/day; exactly-once semantics; decoupled producers/consumers [10]
Schema Enforcement	Confluent Schema Registry (Avro)	Backward-compatible schema evolution; binary serialization $\approx 40\%$ smaller than JSON
Processing Microservices	Python 3.11 + FastAPI	ASGI async handling; native scikit-learn/pandas integration; rapid iteration
Time-Series Database	InfluxDB 2.7 OSS	Write-heavy IoT workload performance; native retention policies; Flux query language; Grafana plugin [7]
ML Framework	scikit-learn 1.4 + XGBoost 2.0	GBT outperforms neural nets on tabular small-feature datasets [19]; sub-100 ms inference without GPU
Container Orchestration	Kubernetes 1.29 (EKS)	HPA elastic scaling; RBAC namespace isolation; rolling deployments [20]
Service Mesh	Istio 1.20	mTLS for all pod traffic; traffic policy; Zipkin tracing
Observability	Prometheus + Grafana	Time-series metrics; pre-built Kafka and Kubernetes dashboards; alert manager integration
CI/CD	GitHub Actions + Helm 3	Automated build, test, push, and rolling deploy

B. Edge Agent Sensor Sampling and Data Schema

Table V specifies the three sensor modalities collected by the mobile agent and their clinical justification.

Table V
EDGE SENSOR SAMPLING PLAN

Signal	Native API	Unit	Interval	Clinical Relevance
Ambient Light	ALS (Sensor.TYPE_LIGHT)	lux	30 s	Lux mismatch is a primary ergonomic trigger [3]
Screen-to-Eye Distance	Proximity / ToF sensor	cm	30 s	Distance <30 cm exponentially increases accommodation strain [3]
Active Screen Time	UsageStatsManager	min	5 min	Session duration is the strongest CVS onset predictor [18]

The Avro schema for a telemetry micro-batch is defined below:

Listing 1. Avro Schema — TelemetryBatch

```

1 {
2   "type": "record",
3   "name": "TelemetryBatch",
4   "namespace": "edu.escuelaing.cvs",
5   "fields": [
6     { "name": "device_uuid", "type": "string" },
7     { "name": "batch_timestamp", "type": "long",
8       "logicalType": "timestamp-millis" },
9     { "name": "readings", "type": {
10       "type": "array",

```

```

11     "items": {
12         "type": "record",
13         "name": "SensorReading",
14         "fields": [
15             { "name": "sensor_type", "type": {
16                 "type": "enum", "name": "SensorType",
17                 "symbols": ["AMBIENT_LIGHT", "PROXIMITY", "SCREEN_TIME"]
18             }},
19             { "name": "value", "type": "float" },
20             { "name": "unit", "type": "string" },
21             { "name": "sampled_at", "type": "long",
22               "logicalType": "timestamp-millis" }
23         ]
24     },
25     { "name": "app_version", "type": "string" },
26     { "name": "consent_hash", "type": "string",
27       "doc": "SHA-256 of user-signed consent document" }
28 ]
29 }
30 }

```

C. Kafka Topic Design

Table VI
KAFKA TOPIC DESIGN

Topic Name	Partitions	Replicas	Purpose
cvs.telemetry.raw	12	3	Receives validated Avro micro-batches from API Gateway
cvs.telemetry.normalized	12	3	Z-scored, anonymized records from Normalizer+Anonymizer microservices
cvs.alerts.immediate	6	3	20-20-20 rule violations requiring real-time push notification
cvs.inference.scores	6	3	CVS Risk Scores produced by the Inference Microservice

Partition count of 12 for the ingestion topics is calibrated to support the expected maximum of 10,000 concurrent device connections per organizational tenant. With Kafka’s documented throughput of millions of events per second across a 3-broker cluster [10], 12 partitions provide a safety margin of approximately 10x the expected peak load.

D. InfluxDB Data Model

Table VII
INFLUXDB MEASUREMENT SCHEMA: SENSOR_READINGS

Element	Name	Type	Description
Tag	device_uuid	String	Pseudonymous device identifier
Tag	org_unit	String	Hashed organizational unit code
Tag	sensor_type	String	AMBIENT_LIGHT, PROXIMITY, SCREEN_TIME
Tag	device_class	String	phone, tablet, laptop
Field	raw_value	Float	Original sensor reading
Field	normalized_value	Float	Z-scored: ($\mu = 0$, $\sigma = 1$)
Field	noised_value	Float	DP-perturbed ($\epsilon = 1.0$)
Field	quality_flag	Integer	0=OK, 1=interpolated, 2=out-of-range
Timestamp	_time	ns (UTC)	Kafka ingestion timestamp

InfluxDB 2.7 was selected over TimescaleDB because the platform’s workload is write-dominant with time-range aggregation queries, where InfluxDB’s line protocol and compaction strategy provide documented throughput advantages [7]. TimescaleDB would be preferred if complex relational joins with user metadata were a primary access pattern—a scenario not present in the current prototype scope.

E. Machine Learning Pipeline

1) *Feature Engineering*: The model trainer constructs a 30-day feature window per pseudonymous UUID:

Listing 2. Flux Query for 30-Day Feature Window

```

1  from(bucket: "cvs_platform")
2  |> range(start: -30d)
3  |> filter(fn: (r) => r["_measurement"] == "sensor_readings")
4  |> filter(fn: (r) => r["device_uuid"] == param_uuid)
5  |> aggregateWindow(every: 1h, fn: mean, createEmpty: false)
6  |> pivot(rowKey: ["_time"],
7         columnKey: ["sensor_type"],
8         valueColumn: "_value")

```

The resulting feature matrix $\mathbf{X} \in \mathbb{R}^{n \times 9}$ contains the features described in Table VIII.

Table VIII
CVS RISK SCORE MODEL: FEATURE MATRIX

#	Feature	Description
1	mean_lux_daily	Mean daily ambient lux level
2	std_lux_daily	Lux standard deviation (lighting variability)
3	mean_proximity_cm	Mean screen-to-eye distance in active sessions
4	min_proximity_cm	Minimum proximity (worst-case posture)
5	total_screen_min	Total daily screen time in minutes
6	max_cont_session_min	Longest continuous session without 20 min break
7	lux_screen_ratio	Ambient lux to estimated screen brightness ratio
8	evening_screen_ratio	Fraction of screen time occurring after 20:00
9	break_compliance_score	Fraction of 20-min intervals with ≥ 3 min break

2) *Model Selection and Training*: XGBoost Gradient Boosted Trees are adopted over neural network alternatives for three reasons: (i) tabular sensor data with a small number of engineered features consistently favors tree-based ensembles in low-resource settings [19]; (ii) gradient-boosted models produce interpretable feature importance scores suitable for clinical validation; (iii) inference latency is in the millisecond range without GPU hardware, compatible with the sub-100 ms SLO defined in Section XII.

Training follows a stratified 5-fold cross-validation strategy. The CVS Risk Score label $y \in \{0,1\}$ is constructed synthetically: a user is labeled high-risk ($y = 1$) if `break_compliance_score` < 0.40 AND `max_cont_session_min` > 45 simultaneously, consistent with the physiological thresholds from Sheppard and Wolffsohn [2].

Figure 6 presents the complete ML pipeline diagram.

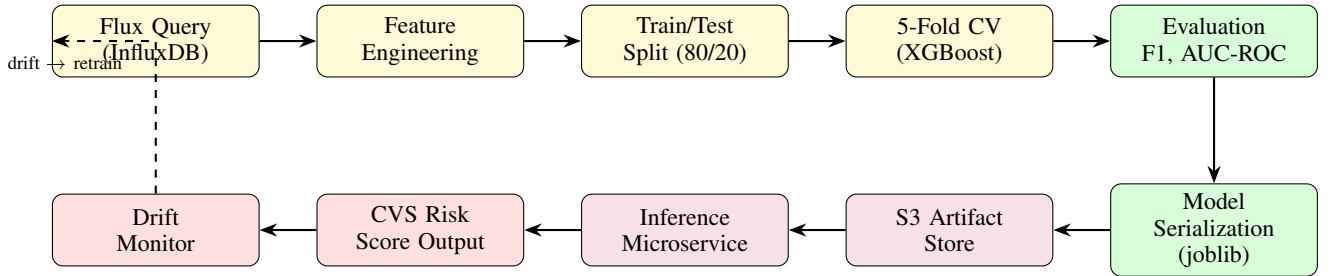


Figure 6. Machine Learning Pipeline: from InfluxDB query to CVS Risk Score inference.

F. REST API Specification

Table IX
REST API ENDPOINT SPECIFICATION (V1)

Method	Endpoint	Auth	Description
POST	/v1/telemetry/ingest	JWT	Accepts Avro micro-batch; publishes to <code>cvs.telemetry.raw</code>
GET	/v1/scores/{uuid}/current	JWT	Returns current CVS Risk Score and top-3 feature importances
GET	/v1/org/{org_id}/risk-summary	JWT + RBAC	Returns organizational risk distribution (P25, P50, P75, P95)
POST	/v1/alerts/acknowledge	JWT	Marks alert acknowledged; suppresses duplicate notifications for 4 h
GET	/v1/health	None	Kubernetes liveness and readiness probe endpoint
DELETE	/v1/users/{uuid}	JWT + RBAC	GDPR right-to-erasure: triggers pseudonymous UUID deletion across all stores

G. Kubernetes Deployment Manifests and Scaling Policy

Listing 3. Helm Values: HPA and Persistence Configuration

```

1 # values.yaml (excerpt)
2 normalizer:
3   replicaCount: 2
4   hpa:
5     enabled: true
6     minReplicas: 1
7     maxReplicas: 8
8     targetCPUUtilizationPercentage: 60
9
10 inferenceSvc:
11   replicaCount: 1
12   hpa:
13     enabled: true
14     minReplicas: 1
15     maxReplicas: 4
16     targetCPUUtilizationPercentage: 70
17
18 kafka:
19   replicaCount: 3
20   persistence:
21     enabled: true
22     storageClass: "gp3"
23     size: "50Gi"
24
25 influxdb:
26   persistence:
27     enabled: true
28     storageClass: "gp3"
29     size: "100Gi"
30   retentionDays: 90

```

H. Sequence Diagram: Alert Generation Lifecycle

Figure 7 presents the sequence diagram for the complete alert generation lifecycle from sensor reading to push notification delivery.

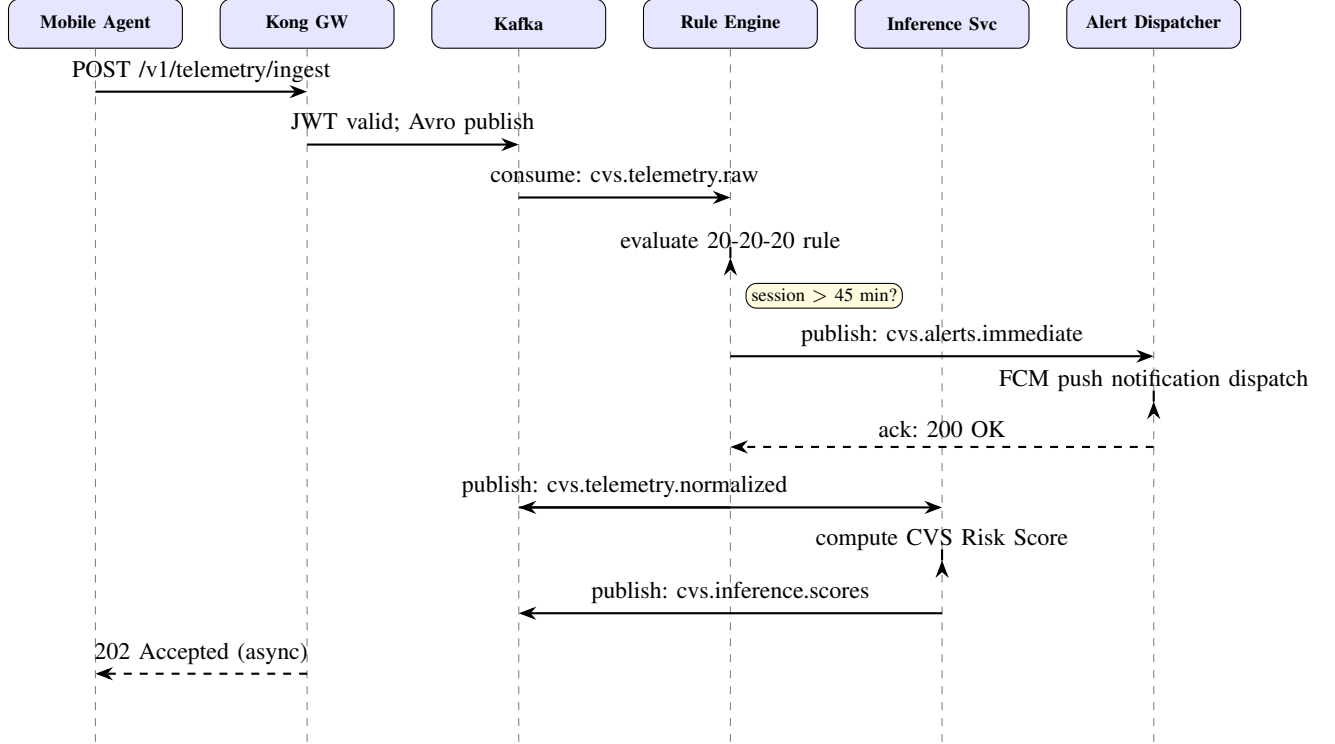


Figure 7. Sequence Diagram: telemetry ingestion and alert generation lifecycle.

VII. APPLIED MICROSERVICES DESIGN PATTERNS

The CVS platform deliberately applies four established microservices design patterns to address the distributed systems challenges inherent in high-throughput IoT telemetry processing [28], [29]. This section documents each pattern, its motivation, and its concrete realization in the prototype.

A. Database-per-Service Pattern

Motivation. In a microservices architecture, sharing a single database across multiple services introduces tight coupling: a schema change in one service can break another, and a database overload caused by one service degrades all others [29].

Realization. Each microservice in Layer 3 operates on its own data boundary. The Rule Engine operates exclusively on in-memory Kafka message state without persisting to any database. The Normalizer and Anonymizer Microservices each maintain stateless transformation logic and write their outputs exclusively to the next Kafka topic in the pipeline—they never directly query InfluxDB. Only the persistence microservice (InfluxDB writer) has write access to the time-series store. The Inference Microservice has read-only access to InfluxDB and read-only access to the S3 model artifact store. This strict access boundary prevents cross-service data coupling and allows each service to be deployed, scaled, and replaced independently.

B. Event Sourcing with Kafka as Immutable Log

Motivation. Traditional CRUD-based architectures store only the current state of an entity, making it difficult to reconstruct historical state, debug processing errors, or replay events after a consumer failure [28].

Realization. Apache Kafka’s partitioned commit log serves as the event source for the entire telemetry processing pipeline. All state changes—from raw sensor reading to normalized record to inference score—are captured as a sequence of immutable events in the Kafka topics. The configurable log retention period (default: 7 days for `cvs.telemetry.raw`) allows any consumer to replay missed events from any point in the recent past without requiring re-ingestion from mobile agents. This property is critical for the ML training pipeline: if a model trainer job fails mid-execution, it can resume from the last committed Kafka offset rather than losing all consumed records. InfluxDB then acts as the materialized view of the Kafka event stream, providing efficient time-range query access to the processed sensor history.

C. CQRS (Command Query Responsibility Segregation)

Motivation. The telemetry ingestion path (writes) and the analytics dashboard path (reads) have fundamentally different performance characteristics: writes are high-throughput, low-latency, and structurally simple (raw sensor records), while reads are complex aggregations over time ranges across millions of records [28]. Forcing both through the same data layer would create performance contention and architectural coupling.

Realization. The platform implements CQRS at the Kafka topic level. The write model is the `cvs.telemetry.raw` topic, optimized for high-throughput sequential append. The read model is InfluxDB, which receives only processed, anonymized records via the `cvs.telemetry.normalized` topic and is optimized for time-range aggregation queries. The REST API enforces this separation explicitly: the `POST /v1/telemetry/ingest` endpoint (command path) routes exclusively to Kafka, while the `GET /v1/scores/{uuid}/current` and `GET /v1/org/{org_id}/risk-summary` endpoints (query path) route exclusively to InfluxDB and the Inference Microservice. The read and write paths never share a processing component, enabling independent horizontal scaling of each path.

D. Circuit Breaker Pattern (via Istio)

Motivation. In a distributed system with many inter-service calls, a single failing service can cascade failures to all dependent services through repeated connection attempts and timeout accumulation—a problem known as cascading failure [29]. The Circuit Breaker pattern prevents this by monitoring failure rates and temporarily blocking calls to a failing service, allowing it to recover.

Realization. The Istio service mesh enforces Circuit Breaker behavior through its `DestinationRule` custom resources. Table X specifies the circuit breaker configuration for the three critical inter-service communication pairs.

Table X
CIRCUIT BREAKER CONFIGURATION VIA ISTIO DESTINATIONRULE

Service Pair	Consecutive Errors	Ejection Time	Behavior on Open
Kong GW → Kafka	5	30 s	Return 503 to mobile agent; agent retries after TTL
Normalizer → InfluxDB	3	60 s	Pause normalization; buffer events in Kafka
Inference Svc → S3	3	120 s	Serve stale model from memory cache

The stale model cache for the Inference Microservice is a particularly important resilience mechanism: if the S3 store becomes unavailable during a model artifact refresh, the inference service continues serving the last successfully loaded model version without interruption, maintaining the `cvs.inference.scores` output topic with a small accuracy penalty acknowledged in the risk score metadata field.

VIII. CI/CD PIPELINE AND DEVOPS ARCHITECTURE

A. Pipeline Overview

The CI/CD pipeline follows the GitOps principle: all infrastructure configuration is version-controlled in the same GitHub repository as application code, and deployments are triggered exclusively by pull-request merges into protected branches. The pipeline is implemented using GitHub Actions workflows and Helm 3 chart-based deployments to the EKS cluster. Figure 8 illustrates the pipeline stages.

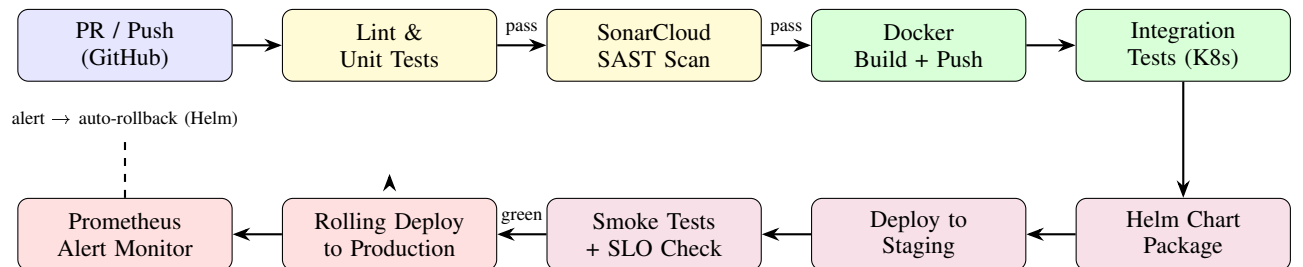


Figure 8. CI/CD Pipeline: from pull request to production rolling deployment.

B. Pipeline Stage Specifications

1) *Lint and Unit Tests*: Each microservice’s GitHub Actions workflow executes `pylint` (Python code style), `mypy` (static type checking), and `pytest` (unit tests with $\geq 80\%$ line coverage requirement). Test failures block the pipeline at this stage, preventing untested code from reaching the container build phase.

2) *SAST Scan with SonarCloud*: The static application security testing (SAST) stage runs a SonarCloud scan against all microservice codebases. This stage is consistent with the DevSecOps posture required for handling personal behavioral data under GDPR: any new code introducing a critical security vulnerability (CWE classification) blocks the PR merge. The SonarCloud quality gate is configured with a coverage delta threshold of $\geq 75\%$ on new code, a maximum of zero critical bugs, and zero security hotspots in the *to review* state.

3) *Docker Multi-Stage Build*: Each microservice uses a two-stage Dockerfile: (i) a *builder* stage based on `python:3.11-slim` that installs dependencies and runs tests; (ii) a *runtime* stage that copies only the compiled application artifact, reducing the final image size by approximately 60% relative to a single-stage build. Images are pushed to Amazon ECR with both a version tag (`v1.2.3`) and a SHA tag (`sha-abc1234`) for immutable deployment references.

4) *Integration Tests on Kubernetes*: Integration tests are executed against a minimal Kubernetes namespace (`ns:test`) spun up in the CI pipeline using the same Helm chart as production, but with reduced replica counts. Tests validate: (i) the Kafka producer-consumer roundtrip latency; (ii) the InfluxDB write-read roundtrip; and (iii) the Inference Microservice HTTP response on a known test payload.

5) *Staging Deployment and Smoke Tests*: The Helm chart is deployed to the `ns:staging` namespace with production configuration values. Smoke tests execute the full telemetry ingestion path with a synthetic device UUID, verifying end-to-end data flow from API Gateway through InfluxDB to Risk Score output. SLO compliance is checked against the P99 latency target.

6) *Production Rolling Deployment and Auto-Rollback*: Production deployments use Kubernetes rolling update strategy (`maxUnavailable: 0`, `maxSurge: 1`) to ensure zero-downtime releases. If Prometheus alerts detect an error rate exceeding 1% within 5 minutes of a new deployment, the GitHub Actions workflow automatically executes `helm rollback` to the previous chart revision.

IX. OBSERVABILITY AND MONITORING ARCHITECTURE

A. Observability Pillars

The observability architecture is structured around the three industry-standard pillars: metrics (Prometheus), logs (structured JSON to CloudWatch), and traces (Zipkin via Istio). Together, these pillars provide full operational visibility into the platform’s health and performance at all times.

B. Prometheus Metrics per Microservice

Each FastAPI microservice exposes a `/metrics` endpoint in Prometheus format via the `prometheus_fastapi_instrumentator` library. Table XI specifies the custom application metrics defined for each service.

Table XI
CUSTOM PROMETHEUS METRICS PER MICROSERVICE

Microservice	Metric Name	Description
Rule Engine	<code>cvs_rule_violations_total</code>	Counter: total 20-20-20 threshold breaches detected
Normalizer	<code>cvs_normalization_latency_hist</code>	Histogram: per-batch normalization processing time
Anonymizer	<code>cvs_dp_noise_applied_total</code>	Counter: total records with Laplace noise applied
Inference Svc	<code>cvs_risk_score_value</code>	Gauge: distribution of CVS Risk Scores in the last 5 min
Inference Svc	<code>cvs_model_version</code>	Gauge: currently loaded model artifact version
Alert Dispatcher	<code>cvs_alerts_dispatched_total</code>	Counter: total FCM push notifications dispatched

C. Grafana Dashboard Design

Three Grafana dashboards are provisioned in the `ns: data & ml` namespace:

- 1) **Platform Operations Dashboard**: Kafka consumer lag per partition, API Gateway request rate and error rate, HPA replica counts per microservice, InfluxDB write throughput, and P99 ingestion latency. This dashboard is the primary on-call tool for the IT/DevOps Administrator.

- 2) **Organizational Health Dashboard:** CVS Risk Score distribution across the organizational unit (histogram), percentage of high-risk devices by department (time series), break compliance trends (weekly), and alert acknowledgment rate. This dashboard is the primary interface for the Organizational Health Officer.
- 3) **ML Model Performance Dashboard:** Model artifact version timeline, risk score statistical distribution drift (KL-divergence over rolling 7-day windows), feature importance evolution across model versions, and inference latency P50/P95/P99.

D. Distributed Tracing

The Istio service mesh instruments all inter-pod HTTP/gRPC calls with Zipkin-compatible distributed tracing headers. Each telemetry ingestion request generates a trace spanning all five processing hops: Kong Gateway → Kafka Producer → Normalizer → Anonymizer → InfluxDB Writer. Trace sampling is set to 1% in production (to limit overhead) and 100% in staging. The trace data is exported to a Tempo backend and visualized in Grafana through the TraceQL query interface, allowing the identification of latency outliers and slow spans within the processing pipeline.

X. DATA GOVERNANCE AND COMPLIANCE FRAMEWORK

A. GDPR Article-by-Article Compliance Mapping

Table XII maps the specific GDPR articles applicable to the platform’s data collection and processing activities to the concrete architectural mechanisms that ensure compliance [12].

Table XII
GDPR COMPLIANCE MAPPING

GDPR Article	Principle	Architectural Mechanism
Art. 5(1)(a)	Lawfulness, fairness, transparency	On-device consent manager displays data collection purpose and scope before first telemetry transmission
Art. 5(1)(b)	Purpose limitation	Telemetry collected exclusively for CVS risk scoring; secondary use contractually prohibited in OHO terms-of-service
Art. 5(1)(c)	Data minimization	Only three sensor signals collected; no camera, microphone, or location data
Art. 5(1)(e)	Storage limitation	InfluxDB 90-day retention policy automatically prunes older records; S3 model artifacts expire after 180 days
Art. 17	Right to erasure	DELETE /v1/users/{uuid} triggers deletion of pseudonymous UUID from InfluxDB, Kafka compaction markers, and S3 metadata
Art. 25	Privacy by design	Differential privacy applied at ingestion; no raw PII persisted; pseudonymous UUID mapping key held by organization only
Art. 32	Security of processing	TLS 1.3 in transit; AES-256 at rest; mTLS pod-to-pod; SQLCipher on device

B. Data Lifecycle and Lineage

Figure 9 illustrates the complete data lifecycle from collection to deletion, showing data transformations and access controls at each stage.

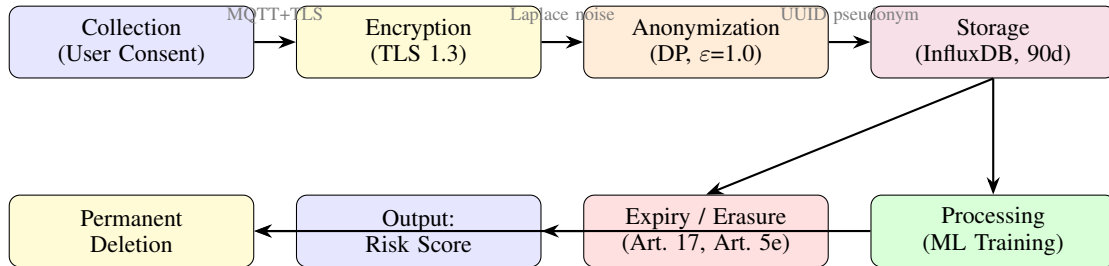


Figure 9. Data Lifecycle Diagram: from collection to permanent deletion.

C. Consent Management Flow

On first application launch, the mobile agent presents the user with a consent dialog that explicitly describes: (i) the three sensor signals to be collected; (ii) the 5-minute micro-batch transmission interval; (iii) the organizational unit that will receive anonymized risk dashboard data; (iv) the 90-day retention period; and (v) the right to withdraw

consent at any time via the `DELETE /v1/users/{uuid}` endpoint. User consent is recorded as a SHA-256 hash of the consent document version and timestamp, stored in the `consent_hash` field of each Avro micro-batch. This hash provides an immutable audit trail demonstrating that each telemetry record was collected under valid consent, supporting GDPR Article 7 accountability requirements.

XI. ARCHITECTURE DECISION RECORDS (ADRS)

This section documents five principal architectural decisions as formal ADRs, following the lightweight Nygard template [27]: Title, Status, Context, Decision, Consequences.

A. ADR-001: Apache Kafka as the Central Messaging Backbone

Status: Accepted. **Date:** 2026-03-01.

Context: The platform must ingest continuous telemetry micro-batches from tens of thousands of mobile devices per organizational tenant. Peak hours generate burst loads that would saturate a synchronous HTTP backend. The Rule Engine, Normalizer, and Anonymizer must scale independently.

Decision: Apache Kafka 3.6 is adopted as the event streaming backbone. Kafka decouples producers from consumers, supports exactly-once semantics, and scales to billions of messages per day with sub-millisecond latency [10]. A 3-broker replica configuration provides partition-level fault tolerance. AWS MSK (Managed Streaming for Kafka) is used in production to reduce operational burden.

Consequences — Positive: Independent consumer scalability; event replay for debugging; exactly-once delivery guarantee. **Consequences — Negative:** Operational complexity of broker cluster; KRaft quorum configuration required.

B. ADR-002: InfluxDB over TimescaleDB for Time-Series Persistence

Status: Accepted. **Date:** 2026-03-01.

Context: The platform must persist high-throughput continuous sensor readings with efficient time-range query capability for ML feature engineering.

Decision: InfluxDB 2.7 OSS is selected. It exhibits superior performance for write-heavy IoT metric workloads and provides native 90-day data retention policy support, simplifying GDPR Article 5(1)(e) compliance [7].

Consequences — Positive: Lower write amplification; Flux query language optimized for time-series aggregations; native Grafana data source plugin. **Consequences — Negative:** Weaker SQL ecosystem; migration to TimescaleDB recommended if JOIN operations with relational metadata become a primary pattern.

C. ADR-003: XGBoost Gradient Boosted Trees for CVS Risk Scoring

Status: Accepted. **Date:** 2026-03-05.

Context: The Inference Microservice must produce CVS Risk Scores with sub-100 ms latency without GPU hardware, using 9 engineered features.

Decision: XGBoost 2.0 Gradient Boosted Trees are adopted. Tree ensembles outperform neural networks on small-feature tabular datasets [19], and provide interpretable feature importances. Inference requires no GPU and completes in <10 ms on CPU-optimized pods.

Consequences — Positive: Fast inference; interpretable outputs; no GPU cost. **Consequences — Negative:** Tree models may underfit complex temporal patterns; LSTM sequential models are designated as a future evaluation target.

D. ADR-004: Differential Privacy ($\epsilon = 1.0$) at the Anonymizer Layer

Status: Accepted. **Date:** 2026-03-05.

Context: The platform collects behavioral data classified as personal data under GDPR. Raw sensor readings transmitted to the cloud create re-identification risk.

Decision: The Anonymizer Microservice applies Laplace-distributed noise with privacy budget $\epsilon = 1.0$. This configuration satisfies GDPR compliance while maintaining utility loss below 5 percentage points [11], [23]. Device identifiers are replaced by pseudonymous UUIDs at first consent.

Consequences — Positive: Mathematical privacy guarantee; GDPR compliance; enables population-level model training without raw data exposure. **Consequences — Negative:** Introduces noise in individual readings; ϵ must be re-tuned per deployment jurisdiction.

E. ADR-005: Kong API Gateway for Authentication and Rate Control

Status: Accepted. **Date:** 2026-03-08.

Context: All inbound telemetry requests must be authenticated, rate-limited to prevent DDoS from compromised devices, and provide a single auditable entry point.

Decision: Kong Gateway 3.x enforces JWT/OAuth2 validation using tokens signed by Keycloak. Microservices validate signatures locally (stateless JWT pattern) without contacting the IdP on every request [9]. mTLS between pods is enforced by Istio.

Consequences — Positive: Centralized authentication; rate limiting; observability. **Consequences — Negative:** Kong is a potential single point of failure; mitigated by deploying 2+ Kong replicas behind the Kubernetes load balancer.

XII. QUALITY ATTRIBUTES AND SERVICE LEVEL OBJECTIVES

A. Quality Attribute Scenarios

Following the Architecture Tradeoff Analysis Method (ATAM), four primary quality attributes are defined with measurable scenarios and target SLOs, as shown in Table XIII.

Table XIII
QUALITY ATTRIBUTE SCENARIOS AND TARGET SLOS

Quality Attr.	Scenario	Target SLO	Architectural Mechanism
Availability	System continues processing telemetry during a single Kafka broker failure	$\geq 99.5\%$ uptime/month	3-replica Kafka; K8s liveness probes; HPA
Scalability	System sustains 10,000 concurrent device connections with <500 ms ingestion latency	P99 latency ≤ 500 ms under 10k producers	Kafka partitioning; stateless microservices; HPA
Security	No raw PII persisted; all inter-service communication encrypted	Zero raw PII in InfluxDB; 100% mTLS inter-pod	Anonymizer DP; Istio mTLS; InfluxDB AES-256 at rest
ML Accuracy	CVS Risk Score correctly classifies high-risk users	F1-score ≥ 0.75 ; AUC-ROC ≥ 0.80	XGBoost; 5-fold CV; 9-feature engineering
ML Inference	Inference Microservice responds within latency bound	P99 inference latency ≤ 100 ms	XGBoost CPU inference; HPA; stale model cache

B. Scalability Analysis

The platform’s horizontal scalability stems from three compounding mechanisms. First, Kafka’s partitioned log allows consumer throughput to scale linearly with partition count: doubling partitions from 12 to 24 approximately doubles parallel consumer capacity without producer code changes [10]. Second, the Kubernetes HPA adds processing pods when CPU utilization exceeds the configured threshold, providing elastic burst capacity aligned with empirical findings from healthcare Kubernetes deployments [20]. Third, InfluxDB’s internal compaction and sharding maintain query performance as data volume grows, with documented advantages for high-cardinality IoT write workloads [7].

A theoretical back-of-envelope capacity estimate: at 10,000 concurrent devices, each transmitting one Avro micro-batch (approximately 1.2 KB binary) every 5 minutes, the aggregate ingestion throughput is $10,000 \times 1200/300 \approx 40$ KB/s, well within Kafka’s documented throughput envelope of hundreds of MB/s per broker.

C. Privacy Compliance Analysis

GDPR compliance is achieved through a defense-in-depth privacy stack: (i) on-device consent management (Art. 7); (ii) TLS 1.3 in transit (Art. 32); (iii) Laplace differential privacy noise at the Anonymizer layer (Art. 25, 32); (iv) pseudonymous UUID mapping (Art. 4(5)); and (v) automated data retention expiry at 90 days (Art. 5(1)(e)). This layered approach is consistent with the principle that combining differential privacy with secure aggregation creates sufficient security to satisfy GDPR legal requirements in healthcare data contexts [11], [13], [23].

XIII. ECONOMIC IMPACT ANALYSIS

A. Organizational Cost-Benefit Model

The economic case for deploying the CVS platform at the organizational level can be modeled around the productivity recovery value generated by reduced CVS incidence. Based on the Deloitte/AOA report [16], an average desk worker affected by unmanaged CVS loses 18.6% of productive output, equivalent to approximately 7.4 hours per working week. For an organization with N knowledge workers and an average annual labor cost of C USD per employee, the annual productivity loss attributable to CVS is approximated as:

$$L_{CVS} = N \times C \times 0.186 \times p_{CVS} \quad (1)$$

where $p_{CVS} = 0.69$ is the global CVS prevalence rate [15].

For a representative Colombian enterprise with 1,000 knowledge workers and an average annual labor cost of \$18,000 USD per employee (consistent with median professional salaries in Bogotá’s technology sector), this yields:

$$L_{CVS} = 1,000 \times 18,000 \times 0.186 \times 0.69 \approx \$2.31 \text{ million USD/year} \quad (2)$$

Conservative estimates suggest that a well-implemented proactive CVS mitigation program can reduce CVS prevalence among enrolled workers by 30–40% through timely break reminders and ergonomic interventions. Applying a conservative 30% reduction factor:

$$\text{Annual Productivity Recovery} \approx \$2.31M \times 0.30 \approx \$693,000 \quad (3)$$

B. Platform Operating Cost Estimate

Table XIV estimates the monthly cloud infrastructure cost for the prototype deployment configuration supporting 1,000 concurrent devices on AWS EKS.

Table XIV
ESTIMATED MONTHLY CLOUD INFRASTRUCTURE COST (AWS, 1K DEVICES)

Component	Estimated Cost/Month	Notes
EKS Cluster (3 node pools)	\$220	t3.medium instances, 6 nodes total
AWS MSK (Kafka, 3 brokers)	\$180	kafka.t3.small, 50 GB storage/broker
InfluxDB (StatefulSet, gp3)	\$45	100 GB gp3 SSD persistent volume
S3 (model artifacts)	\$5	<10 GB model store
Kong Gateway (ELB)	\$30	Application Load Balancer
Prometheus + Grafana	\$20	Single-replica; shared node
GitHub Actions (CI/CD)	\$15	Paid minutes for private repo
Total	\$515/month	(\$6,180/year)

The annual ROI for the 1,000-employee scenario is therefore approximately:

$$\text{ROI} = \frac{\$693,000 - \$6,180}{\$6,180} \approx 111\times \quad (4)$$

This result indicates a strongly positive economic case for enterprise deployment, with infrastructure costs representing less than 1% of the projected productivity recovery value. It is important to note that these estimates carry uncertainty due to the assumptions on intervention effectiveness and the Colombian salary baseline used; the third-phase evaluation will refine these estimates based on empirical measurement of the system’s actual break-compliance improvement rates.

XIV. RISK ANALYSIS AND MITIGATION

Table XV identifies the principal technical and operational risks associated with the prototype architecture, their probability and impact assessment, and the mitigation strategy implemented at the architectural level.

Table XV
RISK ANALYSIS MATRIX

Risk	Prob.	Impact	Mitigation Strategy	Residual Risk
Kafka broker failure	Medium	High	3-replica cluster; auto-recovery via K8s liveness probes; MSK managed failover	Low: P99 TTR $\leq$ 30s expected
InfluxDB disk saturation	Low	High	90-day retention policy; Grafana alert at 70% disk utilization; gp3 auto-scaling	Very low
ML model drift (concept drift)	Medium	Medium	Rolling 7-day KL-divergence monitoring on risk score distribution; automatic nightly retraining Cron-Job	Medium: requires labeled ground truth
UsageStatsManager permission denial	Medium	Medium	Alternative foreground activity detection via <code>ActivityManager</code> ; documented as fallback path	Low: degrades screen time accuracy only
GDPR right-to-erasure latency	Low	High	DELETE endpoint triggers async deletion across InfluxDB, Kafka compact, S3; completion SLO of 30 days per GDPR Art. 12	Low: GDPR allows 30-day window
Kong Gateway DDoS from compromised devices	Low	High	Per-device rate limiting (100 req/min); IP-based Kong rate plugin; circuit breaker (ADR-001)	Low
Synthetic label misalignment with clinical ground truth	High	Medium	Labels derived from clinical thresholds [2]; third-phase evaluation will compare against optometrist-validated survey data	Medium: acknowledged limitation

XV. EVALUATION METHODOLOGY FOR PHASE THREE

The third phase of this project will conduct an empirical evaluation of the deployed prototype against the SLOs defined in Table XIII. The following experimental designs are specified.

A. Load Testing (Scalability SLO)

An Apache JMeter load test will simulate 10,000 concurrent MQTT producers transmitting micro-batches at 5-minute intervals. The test runs for 60 minutes to allow HPA stabilization. Metrics collected: Kafka consumer lag per partition, P50/P95/P99 ingestion latency (Prometheus), and HPA scale-out event count. The Scalability SLO passes if P99 latency remains at or below 500 ms throughout the sustained load period.

B. Fault Injection (Availability SLO)

A controlled fault injection experiment terminates one Kafka broker pod with `kubectl delete pod` during active load at the 20-minute mark of the load test. The time-to-recovery (TTR) for all affected partitions is measured from the deletion event to the restoration of zero consumer lag. The Availability SLO passes if $TTR \leq 30$ seconds, consistent with Kafka’s documented controller-election behavior [10].

C. ML Model Evaluation (ML Accuracy SLO)

The XGBoost model will be trained on a synthetic behavioral dataset of 50,000 user-day records, calibrated to the clinical thresholds of Sheppard and Wolffsohn [2]. The held-out test set (20%) will be evaluated for: F1-score (macro average), AUC-ROC, Precision, and Recall. Feature importance analysis will verify that `max_cont_session_min` and `break_compliance_score` rank among the top three contributors, as expected from the clinical literature.

D. Privacy Audit (Security SLO)

An automated assertion suite will verify that zero raw PII fields appear in any InfluxDB write-path data. A Kolmogorov-Smirnov test will verify that the Laplace noise injection preserves the original sensor distributions within a threshold of $p > 0.05$, confirming that the DP mechanism does not distort statistical utility beyond acceptable bounds.

E. Inference Latency Benchmark (Inference SLO)

The Inference Microservice will be benchmarked under a sustained request rate of 1,000 risk score requests per minute using `locust.io`. The benchmark runs for 10 minutes and records P50, P95, and P99 inference latency. The Inference SLO passes if P99 latency remains at or below 100 ms.

XVI. DISCUSSION

From reactive to proactive. The ML inference pipeline computes a population- calibrated CVS Risk Score reflecting the interaction of multiple behavioral factors, enabling preemptive intervention before symptomatic onset—a capability absent from all existing commercial tools reviewed in the related work.

From edge silos to organizational intelligence. The cloud-native architecture aggregates anonymized behavioral signals across entire organizational units, enabling Health Officers to identify high-risk departments or usage patterns invisible to any individual device’s local analytics. This aligns with the TOGAF Business Architecture objective of providing actionable visibility into the “digital wear” of human capital.

From camera-based to non-invasive proxy sensing. The exclusive reliance on ambient light, proximity, and interaction time avoids the documented privacy rejection associated with continuous camera-based monitoring [4]. The precision trade-off is compensated by operating on 30-day behavioral windows and population-level baselines rather than instantaneous physiological readings.

One acknowledged limitation is the dependency on Android’s `UsageStatsManager` API, which requires a special system permission not uniformly grantable in all enterprise MDM configurations. The alternative foreground activity detection path described in the risk analysis (Table XV) provides a fallback at reduced accuracy. Additionally, the synthetic label construction introduces potential misalignment with ground-truth clinical assessment; this will be partially addressed in the third phase by comparing model predictions against optometrist survey responses from a volunteer participant group.

XVII. CONCLUSIONS AND FUTURE WORK

This article has presented the complete general architecture and prototype design for a distributed cloud-native platform for the proactive mitigation of Computer Vision Syndrome through behavioral telemetry analytics. The principal contributions are:

- 1) A **formally specified four-layer architecture** documented through five complementary IEEE/ISO 42010-aligned architectural views: context, component, data-flow, deployment, and security.
- 2) A **TOGAF ADM-aligned enterprise architecture** view mapping the four TOGAF domains (Business, Data, Application, Technology) to the platform’s structural components, supporting the organizational governance objectives of the course.
- 3) A **prototype technology stack** with technically justified selections: Apache Kafka for event streaming, InfluxDB for time-series persistence, XGBoost for CVS Risk Score inference, Kubernetes/Helm for orchestration, and Kong/Istio for security enforcement.
- 4) **Five Architecture Decision Records (ADRs)** formalizing the most consequential design choices, including the differential privacy configuration, the ML model selection, and the API Gateway strategy.
- 5) **Four applied microservices design patterns** (Database-per-Service, Event Sourcing, CQRS, Circuit Breaker) with concrete realizations in the prototype, increasing resilience, consistency, and operational observability.
- 6) A **CI/CD and observability architecture** covering the complete DevSecOps pipeline from pull request to production deployment with auto-rollback.
- 7) A **GDPR-compliant data governance framework** with Article-by-Article compliance mapping and a formal data lifecycle diagram.
- 8) **Five measurable SLOs** (availability $\geq 99.5\%$; P99 latency ≤ 500 ms under 10k producers; zero raw PII in persistence; $F1 \geq 0.75$; inference P99 ≤ 100 ms) with specified evaluation methodology for Phase 3.
- 9) An **economic impact analysis** demonstrating a theoretical ROI of approximately $111\times$ for a 1,000-employee Colombian enterprise.

The third phase will focus on: (i) deploying the Helm chart to a managed EKS cluster; (ii) executing the load testing, fault injection, and privacy audit experiments; (iii) training the XGBoost model on a synthetic behavioral dataset calibrated to the clinical thresholds of Sheppard and Wolffsohn [2]; and (iv) reporting quantitative quality attribute measurements against all defined SLOs. The long-term vision is a platform that enables organizations to treat CVS not as an individual productivity nuisance but as a measurable, monitorable, and systematically reducible occupational health risk at the enterprise scale.

REFERENCES

- [1] R. A. Elhai, J. D. Elhai, and B. J. Hall, "Future directions in the study of technology addiction," *Addictive Behaviors*, vol. 112, 2021.
- [2] A. L. Sheppard and J. S. Wolffsohn, "Digital eye strain: prevalence, measurement and amelioration," *BMJ Open Ophthalmology*, vol. 3, no. 1, p. e000146, 2018.
- [3] C. Blehm, S. Vishnu, A. Khattak, S. Mitra, and R. W. Yee, "Computer vision syndrome: a review," *Survey of Ophthalmology*, vol. 50, no. 3, pp. 253–262, 2005.
- [4] N. D. Lane et al., "A survey of mobile phone sensing," *IEEE Communications Magazine*, vol. 48, no. 9, pp. 140–150, Sept. 2010.
- [5] A. A. Mutlag, M. K. Abd Ghani, N. Arunkumar, M. A. Mohammed, and O. Mohd, "Enabling technologies for fog computing in healthcare IoT systems," *Future Generation Computer Systems*, vol. 90, pp. 62–78, 2019.
- [6] K. Mahmoud, K. Orsborn, and T. Risch, "Survey on IoT application architectures," *MDPI Sensors*, 2024. [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC11360728/>
- [7] OctaByte, "InfluxDB vs TimescaleDB: Which is Better for Time-Series Data?" Sept. 2025. [Online]. Available: <https://blog.octabyte.io/topics/open-source-databases/influxdb-vs-timescaledb/>
- [8] S. B. Chauhan and R. N. Gore, "SMQTT: A Lightweight Clock Synchronization Algorithm for IoT Devices Using MQTT," in *Proc. IEEE ISPCS*, London, UK, 2023, pp. 1–6.
- [9] Paradigma Digital, "Architecture Patterns in Microservices: Security with JWT," 2024. [Online]. Available: <https://en.paradigmigital.com/dev/architecture-patterns-microservices-security-jwt/>
- [10] Apache Software Foundation, "Apache Kafka 3.6 Documentation," 2024. [Online]. Available: <https://kafka.apache.org/documentation/>
- [11] B. M. Alotaibi et al., "Privacy-Preserving, Regulatory-Grade Real-Time Healthcare AI," *American Journal of Technology*, Dec. 2025. [Online]. Available: <https://gprjournals.org/journals/index.php/ajt/article/view/459>
- [12] European Parliament and Council, "Regulation (EU) 2016/679 — General Data Protection Regulation (GDPR)," 2016.
- [13] V. Berneker et al., "Federated Machine Learning, Privacy-Enhancing Technologies, and Data Protection Laws in Medical Research: Scoping Review," *J. Med. Internet Res.*, vol. 25, p. e41588, 2023.
- [14] H. Ali et al., "Federated Learning in Smart Healthcare: A Comprehensive Review on Privacy, Security, and Predictive Analytics with IoT Integration," *MDPI Electronics*, Dec. 2024. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC11728217/>
- [15] SmartErgo, "December 2024: Digital Eye Strain Prevalence and Mitigation Into 2025," 2024. [Online]. Available: <https://smartergo.com/december-2024-digital-eye-strain-prevalence-and-mitigation-into-2025/>
- [16] American Optometric Association and Deloitte Economics Institute, "The Impact of Unmanaged Excessive Screen Time in the United States," 2023. [Online]. Available: https://www.aoa.org/AOA/Documents/Eye%20Deserve%20More/Cost%20of%20Unmanaged%20Screen%20Time%20Report_FINAL.pdf
- [17] VSP Vision Care and Workplace Intelligence, "2025 Workplace Vision Health Report," 2025. [Online]. Available: <https://workplaceintelligence.com/2024-workplace-vision-health/>
- [18] J. Talens-Estareles et al., "Digital eye strain symptoms worsen during prolonged digital tasks, associated with a reduction in productivity," *Optometry and Vision Science*, Sept. 2024.
- [19] N. Alshurafa et al., "Wearable network for multilevel physical fatigue prediction in manufacturing workers," *PNAS Nexus*, vol. 3, no. 10, p. pgae421, 2024.
- [20] V. V. R. Boda and H. Allam, "Scaling Kubernetes for Healthcare: Real Lessons from the Field," *Int. J. Emerg. Res. Eng. Technol.*, vol. 4, no. 3, pp. 27–34, 2023.
- [21] M. Nygard, "Architecture Decision Records," 2011. [Online]. Available: <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>
- [22] J. Chae et al., "Visual fatigue prediction using classification model based on physiological responses of occupants under office lightings," *Journal of Building Engineering*, Oct. 2024.
- [23] A. A. Mutlag et al., "Privacy preservation for federated learning in healthcare," *Patterns (N.Y.)*, vol. 5, no. 7, p. 100974, July 2024.
- [24] The Open Group, "TOGAF® Standard, 10th Edition," 2022. [Online]. Available: <https://www.opengroup.org/togaf>
- [25] D. Pal, J. London, A. Aakula, and S. Chitta, "Implementing TOGAF for Large-Scale Healthcare Systems Integration," *IoT and Edge Computing Journal*, 2024. Available: <https://thesciencebrigade.com/iotecj/article/view/464>
- [26] X.-K. Yeh, "Unpacking the Structure of the TOGAF Standard: A Strategic Blueprint for Enterprise Architects," *Medium*, Oct. 2025. [Online]. Available: <https://medium.com/@leoyeh.me>
- [27] M. Nygard, "Architecture Decision Records," *Cognitect Blog*, 2011. [Online]. Available: <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>
- [28] C. Richardson, "Microservices Patterns: With Examples in Java," 1st ed. Manning Publications, 2018.
- [29] Atlassian, "7 Essential Microservices Design Patterns," 2024. [Online]. Available: <https://www.atlassian.com/microservices/cloud-computing/microservices-design-patterns>